

基于 CUDA + pybind11 的简易卷积网络框架与 CIFAR-10 分类

2400013080 李玮博

2026 年 1 月 17 日

1 实验背景

深度学习框架通常将张量运算、自动微分与 GPU 加速高度封装。为了理解卷积网络的底层实现流程，本实验在不依赖现成深度学习框架的前提下，使用 CUDA 实现卷积前向与反向传播，通过 pybind11 暴露为 Python 可调用模块，并在 Python 侧完成自动微分与优化器。最终在 CIFAR-10 数据集上完成图像分类任务，验证该最小框架能够完成端到端训练。

2 方法

2.1 整体架构

框架分为三层：（1）CUDA/C++ 底层：实现张量存储、基础算子、卷积与反向传播等；（2）pybind11 绑定层：将 CUDA 算子注册为 Python 可调用接口；（3）Python 前端：构建 Tensor 与计算图，完成自动微分、优化器与模型定义。

2.2 CUDA 卷积前向与反向

卷积实现位于 `csrc/layers/conv.cu`，采用 `im2col` 将输入展开为矩阵，再调用 GPU 矩阵乘法完成卷积前向。反向传播包含两部分：对卷积核的梯度通过 `col` 与输出梯度矩阵相乘得到；对输入的梯度通过转置卷积（`col2img`）重建并累加。该实现支持 `padding` 与 `stride`。

2.3 pybind11 接口

绑定代码位于 `csrc/pybind.cpp`，编译为 `torchofdreaveler.core` 模块。接口以函数形式暴露，例如 `convolve` 与 `convolve_backward`，并在 Python 侧封装为算子类。

2.4 自动微分与优化器

Python 侧 `torchofdreaveler/_core/operators.py` 定义计算图与算子，通过 `compute_gradient_of_variable` 实现反向传播梯度。优化器实现位于 `torchofdreaveler/optim`，提供 SGD 与 Adam。损失函数采用 Softmax 交叉熵，CUDA 实现位于 `csrc/layers/softmax_loss.cu`。

2.5 模型与训练流程

模型示例采用 Lenet 风格的 CNN 与 ResNetSmall(见 `models/lenet.py` 与 `models/resnet.py`), 训练脚本为 `train.py`。数据集加载通过 `torchofdreaveler/data` 封装 `torchvision` 的 CIFAR-10 下载与预处理。

3 实验结果

表 ?? 给出一组可复现实验结果 (来源: `result.md` 中的训练记录)。ResNet20 在 CIFAR-10 上达到约 89.5% 测试准确率; 简单 CNN 基线约 75%。

表 1: CIFAR-10 分类结果

模型	训练轮数	最佳测试准确率	备注
ResNet20	100	89.5%	自定义框架训练
Simple CNN	40	75.1%	参考基线

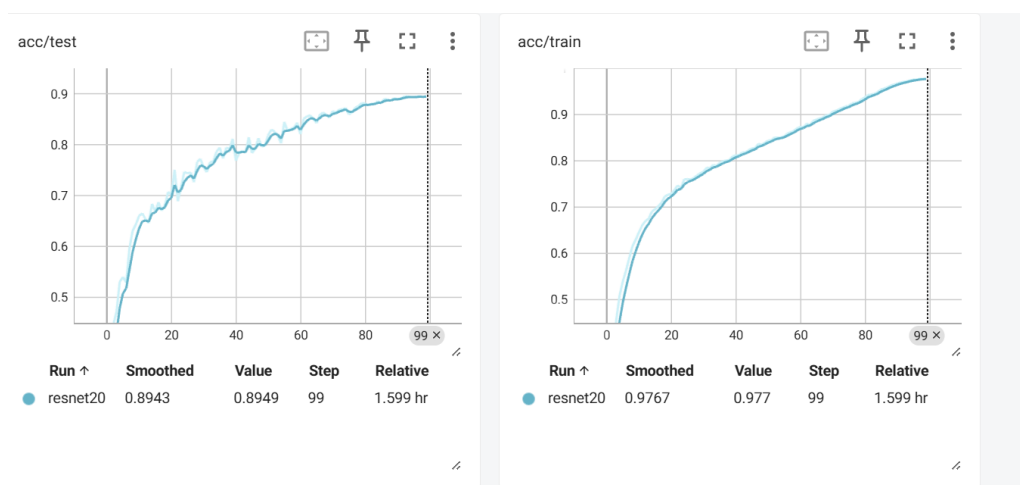


图 1: ResNet20 CIFAR-10 accuracy curve

4 讨论

- (1) **正确性验证:** 卷积前向与反向与 PyTorch 对齐测试已在 `tests/test_operator_parity.py` 中实现, 为算子正确性提供保障。
- (2) **性能瓶颈:** 当前实现以 `im2col + GEMM` 为主, 内存开销较大, 且未对数据布局与融合算子做进一步优化。
- (3) **模型规模与精度:** 简单网络收敛快但精度上限有限, ResNet20 能较好展示框架对更复杂网络的支持能力。

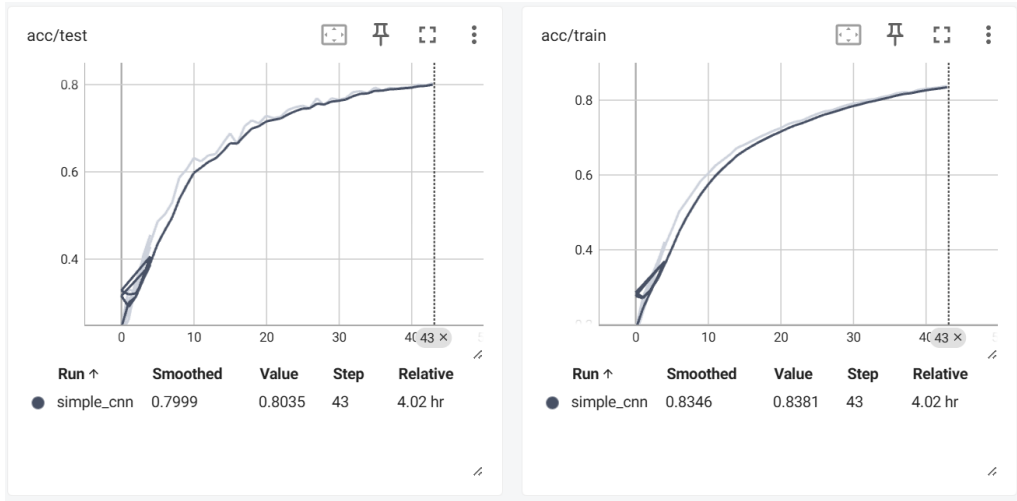


图 2: Simple CNN CIFAR-10 accuracy curve

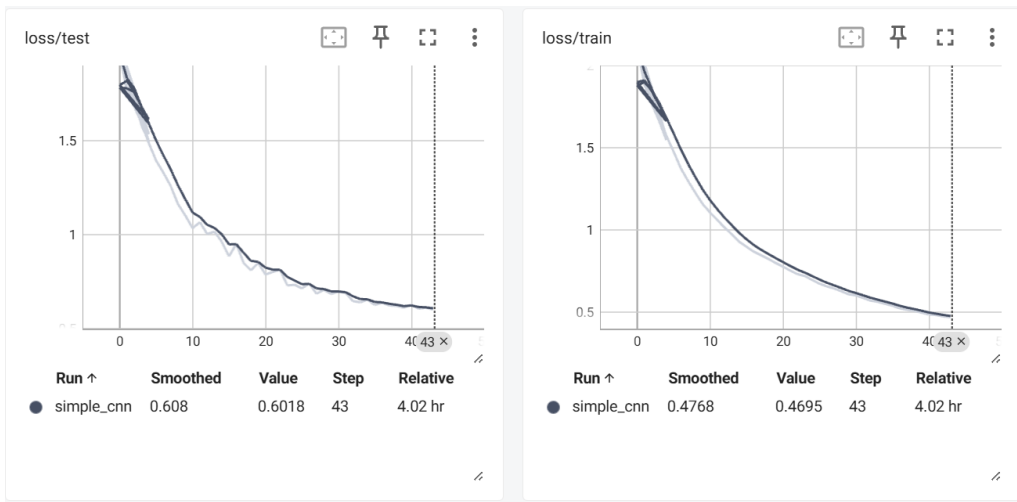


图 3: Simple CNN CIFAR-10 loss curve

5 结论

本实验完成了基于 CUDA 的卷积前向/反向、pybind11 绑定、Python 自动微分与优化器，构建了一个可训练的轻量级卷积网络框架，并在 CIFAR-10 上验证了可用性。后续可进一步优化算子性能、扩展算子覆盖率并完善工程化工具链。

6 如何运行与复现

6.1 环境准备

- Windows + CUDA Toolkit (示例使用 CUDA 13.x)
- Python 3.8+
- 依赖: torch, torchvision, numpy

6.2 pip 安装包

可以直接通过 pip 安装

```
pip install .
```

6.3 包的使用

与 torch 的语言类似，具体细节可以直接参考 `train.py` 文件以及所有 `train` 开始的文件。

且当前实现支持自定义 `sgd` `adam` 等优化器以及 `scheduler` 的参数并支持 `checkpoint` 保存模型，支持使用 `tensorboard` 保存记录并进行可视化

6.4 训练 CIFAR-10

```
python train.py --model lenet --epochs 50 --batch-size 512 --lr 0.1
```

该命令会自动下载 CIFAR-10 至 `./datasets/cifar10`，并在 `checkpoints/` 输出模型权重。若使用 ResNet20 复现实验结果，可运行：

```
python train.py --model resnet20 --epochs 100 --batch-size 512 --lr 0.1
```

7 bonus: 训练 ImageNet

由于 2012 版下载包过大，因此只使用 Tiny ImageNet 来进行训练，我尝试实现了 BatchNorm，但实现细节可能出现问题，算法并行度不够高，导致加上 BN 后 GPU 的使用率极低，而不加 BN 的 `resnet18` 在学习率较大时极易爆炸，经常出现 `loss` 为 `nan` 的情况，因此仍然使用 `resnet20` 去实现，但由于 `resnet20` 的参数量不够使得不太能拟合好模型，且同样由于缺少 BN，收敛速度较慢。结果为训练了 136 个 epoch 后测试准确率为 38.14%

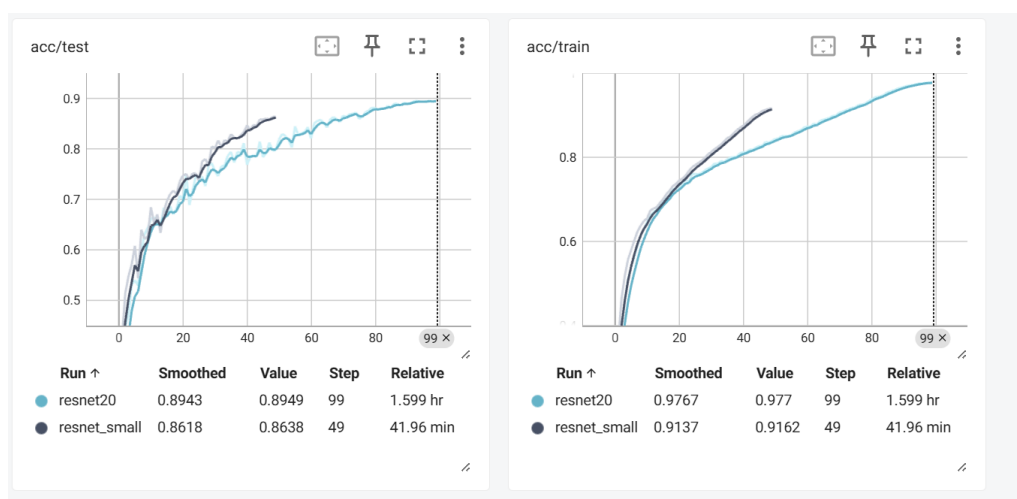


图 4: ResNet20 Tiny ImageNet accuracy curve

具体训练模型与训练参数细节可以参照 `train_tinyimagenet_renet20.py`

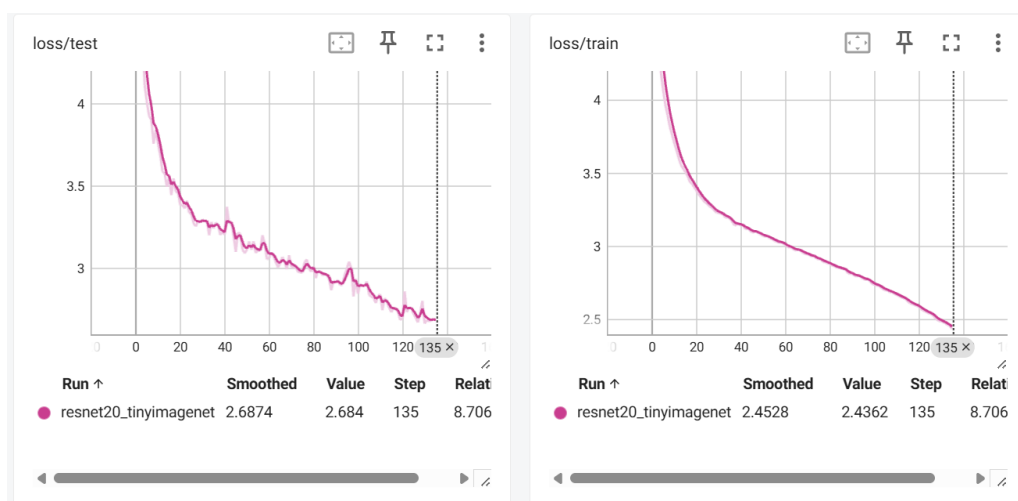


图 5: ResNet20 Tiny ImageNet loss curve